potentially other firmware), it generates a cryptographic signature of the bootloader file. This is achieved by:

- Hashing the bootloader code using a hash function (e.g., SHA-256) to create a fixed-length hash of the code.
- Signing the hash with the private key of the manufacturer or trusted authority, generating the cryptographic signature.
- The signature is then embedded into the bootloader, along with the public key needed for verification. The public key is securely stored within the device's secure storage (e.g., Trusted Platform Module (TPM), Hardware Security Module (HSM), or secure flash memory).

When the device is powered on, the bootloader starts by verifying its own integrity before proceeding further:

- Recalculating the hash of the bootloader code using the same hash function (e.g., SHA-256) used during signing.
- Decrypting the signature using the stored public key and comparing it with the recalculated hash.
- If the decrypted signature matches the calculated hash, it confirms that the bootloader has not been tampered with and is authentic.

The process extends to the firmware as well. The bootloader verifies the integrity of the firmware it is about to load:

- The firmware (including the kernel, filesystem, and other resources) is cryptographically signed by the manufacturer or trusted authority.
- The bootloader checks the signature against the stored public key or a secure element (e.g., TPM or HSM) to ensure that the firmware has not been altered.
- If the firmware passes the integrity check, the bootloader proceeds to load the verified firmware into system memory (RAM), which includes the kernel, filesystem, and necessary components for proper device operation.

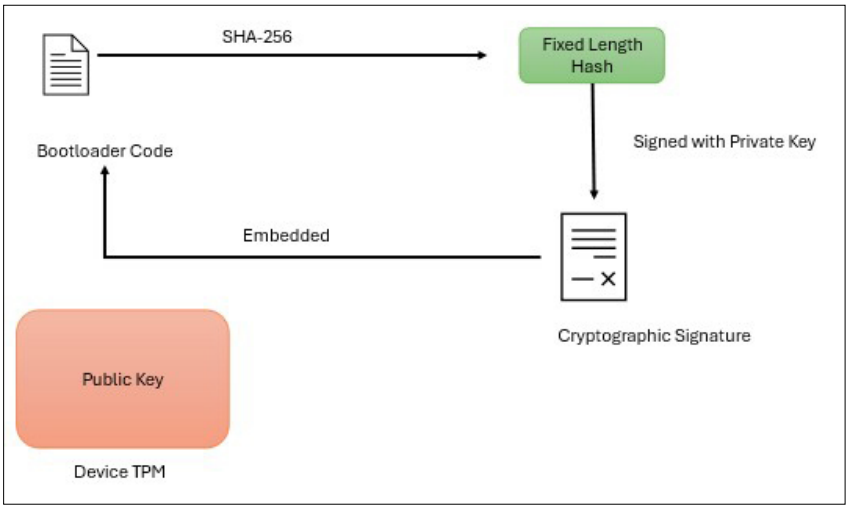

Figure 1: Signature generation

Once the firmware is loaded into memory, it is executed. The device is now operating with trusted code, and any subsequent actions are performed within a secure environment.

Some IoT devices implement runtime integrity checks to monitor the firmware during operation. These checks ensure that no unauthorised code or malicious modifications occur while the device is running, further enhancing its security. Additionally, secure boot enables secure firmware updates**,** where new firmware versions are checked for validity and integrity before being applied. If the secure boot process detects a failure or integrity breach, many devices include a recovery mechanism to revert to a known good state, ensuring that the device can recover from any compromise.

Tamper resistance and security

One of the key benefits of secure boot is its tamper resistance. If an attacker attempts to alter the bootloader or firmware, even slightly, the hash will change, causing the signature verification to fail. This prevents the device from booting with compromised software, protecting it from potential exploitation. The use of trusted public keys stored in secure storage ensures that the verification process is based on an authentic key, making it extremely difficult for attackers to forge a valid signature without access to the private key.

As IoT devices continue to proliferate in both consumer and industrial environments, their security becomes increasingly critical. Secure boot provides an essential layer of protection by ensuring that only trusted software runs during the device's startup process. With cryptographic signatures, manufacturers can safeguard their devices against malicious tampering, preventing unauthorised code execution and protecting users from potential threats. The implementation of secure boot, along with other robust security measures, will play a pivotal role in maintaining the integrity and trustworthiness of IoT devices in the years to come. END

By: Anisha Ghosh

The author is a cybersecurity researcher and an active contributor to the open source communities and repositories. She is interested in developing novel, scalable and secure systems.